



VELS





INSTITUTE OF SCIENCE, TECHNOLOGY & ADVANCED STUDIES (VISTAS)
(Deemed to be University Estd. u/s 3 of the UGC Act, 1956)
PALLAVARAM, THALAMBUR, PERIYAPALAYAM, THIRUVANMIYUR, ADYAR - CHENNAI

FOODLOOP: AN EXPLAINABLE URGENCY-AWARE MATCHING AND COLLECTION PRIORITISATION FRAMEWORK FOR SURPLUS FOOD REDISTRIBUTION

MINI PROJECT REPORT

Submitted by

Sukrish T

Arun A

Sadham Hussian R

Under the guidance of Mr. Akash S

Department of Artificial Intelligence and Machine Learning

in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY CSE –

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

SCHOOL OF ENGINEERING

VELS INSTITUTE OF SCIENCE, TECHNOLOGY AND ADVANCED STUDIES (VISTAS)

CHENNAI

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project entitled “FoodLoop: An Explainable Urgency-Aware Matching and Collection Prioritisation Framework for Surplus Food Redistribution” is the project work of Sukrish T, Arun A and Sadham Hussian R, carried out under the guidance of Mr. Akash S. This certificate is prepared in the format of the reference report; institutional certification and signatures must be completed by the authorised officials.

Certified further that, to the best of the certifying officials’ knowledge, the work reported here has not formed part of any other project report or dissertation on the basis of which a degree or award was conferred earlier on these candidates.

SIGNATURE OF THE HOD	**SIGNATURE OF THE SUPERVISOR**
Dr. K. Ulaga Priya, M.E., Ph.D. Associate Professor & Head Department of AI & ML School of Engineering VISTAS, Chennai – 600 117	Mr. Akash S Project Guide Department of Artificial Intelligence and Machine Learning VISTAS, Chennai – 600 117

Submitted for the Project Work and Viva-Voce examination held on _____ at Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai – 600 117.

INTERNAL EXAMINER	**EXTERNAL EXAMINER**
Signature: _____	Signature: _____

DECLARATION

We hereby declare that the project work entitled “FoodLoop: An Explainable Urgency-Aware Matching and Collection Prioritisation Framework for Surplus Food Redistribution”, submitted to Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai, in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence and Machine Learning), is based on the work carried out by us for the stated project.

The report distinguishes features described in the public demonstration from features that have been tested. The conference paper states that no experiment had been conducted at the time of writing and that the cloud database mode had not been evaluated. We therefore do not claim unverified deployment, performance, or experimental outcomes.

We further declare that this report has not been submitted previously, in part or in full, for the award of any degree, diploma, associateship or fellowship at this or any other university or institution.

NAME	**SIGNATURE**
Sukrish T	
Arun A	
Sadham Hussian R	

ACKNOWLEDGEMENT

We express our sincere gratitude to Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai, and the Department of Artificial Intelligence and Machine Learning, School of Engineering, for providing the academic environment in which this mini project has been prepared.

We thank our project guide, Mr. Akash S, for the guidance associated with this project. We also acknowledge the Head of the Department, Dr. K. Ulaga Priya, and the faculty members of the department. The institutional names and guide details are reproduced from the information supplied for this report; this acknowledgement does not imply any unverified approval or endorsement.

We are grateful to our families and friends for their encouragement. We also acknowledge the cited researchers, institutions and technical documentation that informed the FoodLoop design, including work on food-waste measurement, redistribution platforms, matching methods and data protection.

Finally, we thank everyone who supports efforts to reduce avoidable food waste and improve the coordination of safe surplus-food redistribution.

Sukrish T • Arun A • Sadham Hussian R

ABSTRACT

Surplus food can go unused when donors and recipient organisations cannot identify a suitable recipient and arrange collection before the food's usable deadline. FoodLoop is a web-based prototype and design for coordinating surplus-food donations through listing, discovery, recipient matching, urgency-aware prioritisation, explanation, reservation, pickup tracking and analytics.

The proposed matching method first applies hard eligibility checks so that expired listings and pairs that cannot be collected within the remaining time are excluded. Eligible pairs are scored using four criteria: deadline urgency (base weight 0.40), category fit (0.25), quantity fit (0.20) and collection feasibility (0.15, when reliable travel-time data exist). Missing or incomparable criteria are not treated as favourable or unfavourable; their weights are removed and the remaining weights are rescaled. The system reports assessed-weight coverage and creates explanations from the same criterion values used to calculate each score.

The public demonstration describes donor, recipient and administrator views, donation discovery, matching, reservations, pickup tracking and impact analytics. Its demo data remain in browser storage on the current device. Supabase authentication is described as a cloud mode, but the conference paper does not evaluate a shared database or confirm that the matching specification is identical to deployed code. The paper also states that no experiment has been conducted; therefore, the numerical example and test cases in this report are hand-computed illustrations, not measured results.

FoodLoop's contribution is deliberately limited to an auditable combination of eligibility rules, explained urgency scoring and pickup-outcome records. It is not a food-safety authority, a transport service or a proven operational deployment. Future work includes implementation verification, test execution, secure shared storage, reliable travel-time data, stakeholder-led weight setting and a consent-based field pilot.

Keywords: surplus food redistribution, food rescue, explainable prioritisation, rule-based matching, urgency scoring, pickup tracking, browser storage.

TABLE OF CONTENTS

SECTION	PAGE
ABSTRACT	5
LIST OF FIGURES	8
LIST OF TABLES	9
LIST OF ABBREVIATIONS	10
CHAPTER 1 INTRODUCTION	11
1.1 General	11
1.2 Background and Motivation	11
1.3 Problem Description	11
1.4 Existing System	12
1.5 Limitations of Existing System	12
1.6 Proposed System	13
1.7 Objectives	13
1.8 Scope	13
1.9 Contributions	13
1.10 Modules	14
1.11 Organisation of Report	14
1.12 Summary	14
CHAPTER 2 LITERATURE REVIEW	15
2.1 Food Waste and Redistribution	15
2.2 Digital Food-Waste Platforms	16
2.3 Learning-Based Food Rescue	16
2.4 Allocation and Matching Theory	17
2.5 Multi-Criteria and Explainable Decisions	17
2.6 Recommender Systems	17
2.7 Comparison and Research Gap	17
CHAPTER 3 SYSTEM DESIGN	19
3.1 Existing System	19
3.2 Proposed System	19
3.3 System Architecture	20
3.4 Workflow and UML/DFD Views	21
3.5 Algorithms	22
3.6 Data Flow	22
3.7 System Modules	23
3.8 Hardware Requirements	23
3.9 Software Requirements	23

3.10 Database Design	24
3.11 Component Description	25
3.12 User Interface Design	25
3.13 Feasibility Study	26
3.14 Requirement Specification	26
CHAPTER 4 IMPLEMENTATION AND METHODOLOGY	26
4.1 Methodology	26
4.2 Donation Listing and Discovery	26
4.3 Matching and Prioritisation	27
4.4 Explainability	28
4.5 Reservation and Pickup Tracking	29
4.6 Analytics and Roles	29
4.7 Technology and Implementation Status	30
CHAPTER 5 RESULTS AND DISCUSSION	31
5.1 Evaluation Status	31
5.2 Worked Example	32
5.3 Test Plan	33
5.4 Expected Outcomes	33
5.5 Discussion	33
CHAPTER 6 FOODLOOP WORKSPACE AND DEMONSTRATION	34
6.1 Overview	34
6.2 Main Views	34
6.3 Demo Data and Storage	35
6.4 Safety, Privacy and Fairness	35
CHAPTER 7 CONCLUSION AND FUTURE WORK	36
7.1 Conclusion	36
7.2 Objective Status	40
7.3 Limitations	37
7.4 Future Scope	37
REFERENCES	38
APPENDIX	40

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE
Figure 3.1	FoodLoop architecture (cloud mode not evaluated)	20
Figure 3.2	End-to-end workflow	21
Figure 3.3	Eligibility and urgency-aware ranking flowchart	22
Figure 3.4	Logical data model (proposed schema)	24
Figure 4.3	Reservation and pickup-outcome lifecycle	29
Figure 6.1	FoodLoop workspace views (conceptual)	34

Note: Figures 4.1–4.3, 5.1 and 6.1 are labelled conceptual/illustrative where they do not depict a verified application screenshot. The source material supplied no actual screenshots that could be confirmed as captured from a running session.

LIST OF TABLES

TABLE NO.	TITLE	PAGE
Table 2.1	Comparison of selected research	17
Table 2.2	Comparison of approaches	18
Table 3.1	Feature status classification	19
Table 3.2	Module description	23
Table 3.3	Software requirements and evidence status	23
Table 3.4	Logical data entities and fields	24
Table 3.5	Functional requirements	25
Table 3.6	Non-functional requirements	25
Table 4.1	Donation listing fields	26
Table 4.2	Criterion functions and base weights	27
Table 4.3	Edge-case handling	27
Table 4.4	Technology status	30
Table 5.1	Worked example values	32
Table 5.2	Test cases and expected outcomes	33
Table 7.1	Objectives and evidence status	36
Table A.1	Notation and configuration variables	40

LIST OF ABBREVIATIONS

ABBREVIATION	FULL FORM
AI	Artificial Intelligence
API	Application Programming Interface
DFD	Data Flow Diagram
FAO	Food and Agriculture Organization of the United Nations
FSSAI	Food Safety and Standards Authority of India
HTML	HyperText Markup Language
HTTP/HTTPS	Hypertext Transfer Protocol / Secure
JSON	JavaScript Object Notation
MCDM	Multi-Criteria Decision Making
ML	Machine Learning
RLS	Row-Level Security
SAW	Simple Additive Weighting
UI	User Interface
UNEP	United Nations Environment Programme
UX	User Experience

CHAPTER 1

INTRODUCTION

1.1 General

Food waste is an environmental, economic and social concern. Surplus food may remain edible while being discarded because the people or organisations that could use it are not connected to the donor in time. The Food and Agriculture Organization has documented food loss and waste across supply chains, while the United Nations Environment Programme's Food Waste Index Report 2024 provides updated global measurement and reduction context [1], [2].

FoodLoop addresses one coordination step in that wider problem: helping a recipient identify a suitable donation, understand its priority, reserve it and record what happened at collection. It is a decision-support concept and web-based demonstration, not a food-safety authority or transport operator.

1.2 Background and Motivation

Donation coordination may depend on phone calls, messaging groups and manually maintained lists. These channels can be useful, but the information is often fragmented. A donor may have a limited collection window, while a recipient needs to consider food category, amount, location, capacity and the time at which collection can begin.

FoodLoop proposes a structured listing that records the food name, category, quantity and unit, pickup location, availability start, collect-by deadline, storage information and contact details. The recipient profile records preferred category, maximum quantity and unit, and collection start. Structured inputs make eligibility and prioritisation more consistent and allow a result to be explained.

1.3 Problem Description

The core problem is to rank active donation listings for a recipient without presenting a listing that cannot be collected before its deadline. A naive earliest-deadline ordering can over-prioritise a listing that is already expired or infeasible. A simple category match alone ignores time pressure and recipient capacity.

FoodLoop must also handle missing information honestly. If the recipient has not stated a category preference, the system cannot claim category fit. If travel time is unavailable, feasibility cannot be assessed. A missing value should not silently be converted into either a penalty or full credit. Finally, reservations and pickup outcomes need records so that listed food can be distinguished from collected food.

1.4 Existing System

Existing approaches include donation listing portals, food-sharing platforms, routing or scheduling tools, and informal coordination through calls or messages. Digital platforms can publish offers and support collection confirmation and impact reporting. The FAO AgriTech Observatory describes FoodCloud as an example of a platform through which donors upload surplus offers, charities accept and confirm collections, and operational data support reporting [4].

Research has also considered recommender systems that select volunteers for food rescue [5], [6], and allocation models for distributing donated food under capacity constraints [9]. These approaches demonstrate that the general field is established. FoodLoop does not claim to invent digital donation listings or matching as a whole.

1.5 Limitations of Existing System

Different services address different parts of the donation lifecycle. A listing-only service may not make deadline urgency and recipient fit easy to compare. A routing tool may calculate routes without capturing the food donation, reservation and outcome lifecycle. Informal coordination can make duplicate claims, missed collections and final quantities hard to audit.

These observations describe general limitations and motivation, not a claim that every existing platform lacks the relevant feature. The supplied paper explicitly notes that operating platforms already provide listing, collection confirmation and reporting. Any direct comparison with named services should be supported by a verified feature audit rather than assumption.

1.6 Proposed System

FoodLoop combines donation listing and discovery with a recipient-specific matching process. The proposed sequence is: validate listing data, reject ineligible pairs, calculate assessable criterion values, rescale the weights for the criteria that can be assessed, rank results, show score explanations, and record reservation and pickup outcomes.

The demonstration page describes Overview, Discover food, Saved food, Donate food, Smart matching, My reservations, Pickup tracking, Impact analytics, Admin dashboard and About sections. A role selector offers Recipient, Donor and Administrator views. The paper states that demo-mode changes remain in the browser on the current device. Cloud mode using Supabase authentication is described but was not evaluated [13], [14].

1.7 Objectives

The project objectives are to: (i) capture structured surplus-food listings; (ii) support discovery by category and sorting; (iii) compare active listings with recipient needs and capacity; (iv) rank eligible matches by transparent urgency and fit criteria; (v) explain each ranking using the same values as the score; (vi) support reservations and pickup-status records; and (vii) summarise outcomes through analytics views.

These objectives describe the intended system. Their implementation status differs: the public interface presents several relevant views, but the supplied conference paper reports that the page was read rather than operated and the source code was not inspected. Accordingly, the report does not claim that the algorithm, persistence, permissions or end-to-end workflow passed execution tests.

1.8 Scope of the Project

The scope is limited to a web-based prototype and a documented rule-based method. The system focuses on food category, quantity and unit, availability start, collection deadline, recipient preferences, maximum quantity and collection start. Reliable travel-time data are optional; the public page states that geography and travel time are not scored because verified data are unavailable.

The project does not transport food, inspect or certify food safety, guarantee suitability for consumption, or establish regulatory compliance. It does not claim an operational multi-user cloud deployment, machine-learning model, live notifications or map routing. These are either unverified or future enhancements.

1.9 Contributions

The conference paper identifies four design contributions. First, it specifies eligibility rules and criterion functions for expired listings, invalid quantities, unit mismatches and missing preferences. Second, it defines a missing-data rule that excludes unassessable criteria and rescales remaining weights while reporting assessed coverage. Third, it derives explanations from the values and weights used in the score. Fourth, it proposes reservation and pickup-outcome records that separate listed quantity from collected quantity.

These are design contributions rather than experimentally proven benefits. The literature search was targeted rather than systematic, so the report treats the research gap as a bounded observation about the reviewed sources, not proof that no similar system exists.

1.10 Modules

The conceptual system is divided into six modules: donation input, recipient profile, matching engine, explanation, reservation and pickup tracking, and analytics. Donation input captures and validates offer details. Recipient profiles represent the requirements against which offers are evaluated. The matching engine applies eligibility and scoring rules, while explanation converts the score calculation into human-readable reasons.

Reservation and tracking maintain the state of a donation after a recipient claims it. Analytics aggregates records by period and outcome. Administrator views provide an overview of listings. In the current evidence, these modules are visible or described in the public demonstration, but their complete execution and shared-backend behaviour have not been verified.

1.11 Organisation of the Report

Chapter 1 introduces the problem and objectives. Chapter 2 reviews related work in food waste, digital platforms, matching, multi-criteria decisions and recommendation. Chapter 3 presents the system architecture, data flow, logical data model and requirements. Chapter 4 documents the proposed methodology and feature status. Chapter 5 provides hand-computed examples and the test plan. Chapter 6 describes the demonstration workspace and important limitations. Chapter 7 concludes and identifies future work, followed by references and an appendix.

1.12 Summary

FoodLoop is framed as a practical decision-support design for surplus-food redistribution. Its central design principle is to apply eligibility before urgency scoring and to make each ranking reproducible from visible inputs. The report separates public-page observations from proposed algorithm details and untested implementation claims, preserving the evidence limitations stated in the supplied conference paper.

CHAPTER 2

LITERATURE REVIEW

2.1 Food Waste and Redistribution

Gustavsson et al. prepared the FAO report Global Food Losses and Food Waste: Extent, Causes and Prevention, which documents losses and waste across the supply chain [1]. UNEP's Food Waste Index Report 2024 updates the global context and discusses measuring and reducing food waste [2]. These sources motivate the importance of preventing edible surplus from being wasted, but they do not specify the coordination logic for a single donation.

Redistribution is time-sensitive because prepared and perishable food has a limited usable life. An offer can only provide value if a suitable recipient can collect it in time and handle it appropriately. FoodLoop therefore treats the collection deadline as a hard operational constraint and a ranking signal among eligible listings.

2.2 Digital Food-Waste Platforms

Oroski and da Silva's mini-review examines food-waste-reducing platforms and identifies both their promise and implementation barriers [3]. Digital services can help connect actors, but platform availability alone does not guarantee that food is recovered. Trust, different participant needs and limited empirical evidence remain important considerations.

The FAO AgriTech Observatory's FoodCloud description shows that listing, acceptance, collection confirmation and reporting are established platform capabilities [4]. FoodLoop's proposed contribution must therefore be understood as the specific combination of explicit eligibility, explained urgency scoring and outcome records, rather than a claim to be the first digital food donation platform.

2.3 Learning-Based Food Rescue

Shi, Lizarondo and Fang proposed a recommender system for crowdsourcing food rescue that selects volunteers to notify about rescue opportunities [5]. Later work by Shi and colleagues explored bandit-based data-driven optimisation for volunteer recommendation [6]. This work concerns volunteer engagement and transport participation, which differs from FoodLoop's recipient-side question of which donation listings should be shown and in what order.

Learning-based systems can adapt when reliable interaction histories are available, but a short-lived donation listing may provide little history and new recipients or offers can create a cold-start problem. FoodLoop uses fixed, inspectable rules instead. This increases auditability and reduces dependence on training data, but the weights remain judgement-based and cannot adapt automatically.

2.4 Allocation and Matching Theory

Gale and Shapley introduced a stable matching approach for college admissions and marriage markets [7]. Kuhn described the Hungarian method for assignment problems [8]. These approaches offer foundations for allocating participants when preferences and objective functions are specified. Food donation allocation also involves capacities and competing equity and efficiency objectives.

Sengul Orgut and colleagues modelled the equitable and effective distribution of donated food under capacity constraints [9]. Their work highlights that allocation choices can have distributional consequences. FoodLoop's local ranking method is simpler than a global allocation optimiser: it ranks listings for a recipient and does not guarantee an equitable allocation across all recipients. A future system should evaluate that limitation explicitly.

2.5 Multi-Criteria and Explainable Decisions

The Analytic Hierarchy Process formalises multi-criteria decision-making through structured pairwise judgements [10]. Simple additive weighting provides a more direct approach in which criterion values are combined using weights. Both approaches make contributions inspectable, but weights can reflect subjective choices and should be reviewed with affected stakeholders.

Lee et al. describe WeBuildAI, a participatory framework for algorithmic governance and preference modelling [11]. This work supports the idea that donors and recipient organisations should have a voice in setting priorities. FoodLoop currently specifies fixed weights and shows how they affect a result; it does not claim those weights are validated or socially optimal.

2.6 Recommender Systems

Koren, Bell and Volinsky review matrix-factorisation techniques for recommender systems [12]. Collaborative filtering methods often depend on interaction data and can face cold-start challenges when users or items are new. Food donations are short-lived, and a listing may disappear after a single collection window, making a data-hungry approach less suitable as an initial prototype.

FoodLoop's rule-based approach can produce a score without historical user-item interactions. Each score can be calculated from the current listing and recipient profile. This makes it easier to explain and test manually, though it does not learn preferences from past outcomes or establish that the ranking is more effective than alternative baselines.

2.7 Comparison and Research Gap

The sources reviewed establish that digital donation platforms exist, food-rescue recommenders have been studied, and allocation models address capacity and equity. The bounded gap investigated by FoodLoop is whether a lightweight workflow can combine eligibility rules, urgency-aware ranking, reasons generated from the scoring calculation and recorded collection outcomes.

The comparison below is qualitative and limited to the cited sources. It should not be read as an exhaustive market survey. The conference paper explicitly warns that its literature search was targeted and that the gap is not evidence that no similar system exists.

Table 2.1: Comparison of selected research

Source / area	Primary focus	Relation to FoodLoop
FAO / UNEP [1], [2]	Scale and measurement of food loss/waste	Motivation; not donation-level matching
Oroski & da Silva [3]	Review of food-waste platforms	Platform opportunities and barriers
FoodCloud [4]	Donation offers, collection confirmation, reporting	Shows these platform capabilities already exist
Shi et al. [5], [6]	Volunteer recommendation and optimisation	Learning-based volunteer engagement
Orgut et al. [9]	Equitable/effective allocation under capacity	Highlights allocation trade-offs
Saaty [10]; Lee et al. [11]	MCDM and participatory governance	Supports inspectable criteria and stakeholder input

Table 2.2: Comparison of approaches

Approach	Strength	Limitation for this project
Informal coordination	Flexible and familiar	Information and outcomes may be fragmented
Listing portal	Publishes offers	May leave ranking and urgency comparison to recipients
Route optimisation	Can support travel planning	Does not necessarily cover donation lifecycle
Learning-based recommender	Can adapt from interaction data	Requires useful data; can be hard to audit
FoodLoop proposed rules	Hand-checkable, transparent scores	Weights unvalidated; travel time unavailable; no evaluation

CHAPTER 3

SYSTEM DESIGN

3.1 Existing System

Before a structured workflow is introduced, donors and recipient organisations may coordinate through separate communication channels and manually compare offers. This can make it difficult to identify which listing fits a recipient's food preferences and capacity, whether the collection deadline remains feasible, and whether a previous reservation has already been made.

FoodLoop's reference demonstration presents these functions in one workspace. However, the conference paper says that the public page was read but not operated, and its source code was not inspected. Therefore, this chapter documents the architecture described by the available sources rather than claiming that every module has been verified at runtime.

3.2 Proposed System

The proposed architecture has a browser interface, client-side logic and a data layer. The interface offers donor, recipient and administrator views. Client-side logic validates inputs, filters ineligible donation-recipient pairs, calculates scores and creates reasons. Demo mode stores changes in browser storage on the same device. Cloud mode with Supabase authentication is described in the page, but a shared database and access policies were not evaluated.

Table 3.1 distinguishes interface features that are visible or described from algorithm details proposed in the paper and cloud capabilities not evaluated. This classification is essential to avoid presenting design intent as implementation evidence.

Table 3.1: Feature status classification

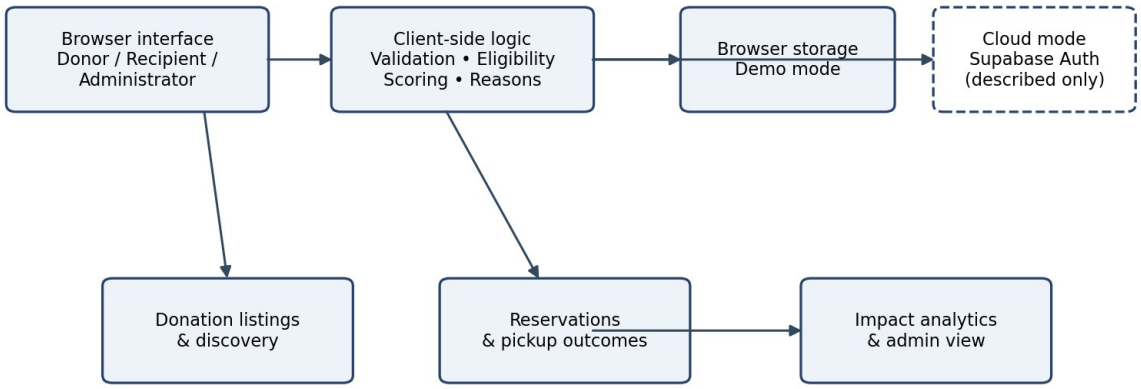
Feature	Status supported by supplied sources	Report treatment
Donation form and discovery	Seen in public UI; not tested	Described as visible, not execution-verified
Matching list and displayed weights	Seen in UI; code not inspected	Rule details treated as proposed specification
Eligibility and duplicate-reservation rules	Not verified	Documented as design requirements
Reservations and pickup tracking	Seen in UI; not tested	Workflow described, not test-confirmed
Analytics and report download	Seen in UI; not tested	Feature status remains unverified
Browser storage	Page states local browser persistence	Not shared or secure; not tested
Supabase Auth / shared database	Cloud mode described; database not evaluated	Not claimed as deployed or validated
Maps, travel-time scoring, notifications	Not implemented / not found in supplied evidence	Future work

3.3 System Architecture

Figure 3.1 summarises the system described by the public demonstration and paper. The solid path represents the browser interface and client-side logic connected to demo-mode browser storage. The dashed cloud box is intentionally distinguished because Supabase authentication is described in the sign-in interface, while the database behind that mode is not evaluated.

The logical architecture is divided into presentation, processing and persistence concerns. Presentation includes listing, discovery, matching, reservations, pickup tracking and analytics. Processing includes field validation, eligibility checks, score calculation, explanation generation and sorting. Persistence in the demonstrated mode is browser-local. A shared deployment would require a properly configured database, authentication, access policies and concurrency safeguards.

Figure 3.1: FoodLoop architecture (cloud mode not evaluated)

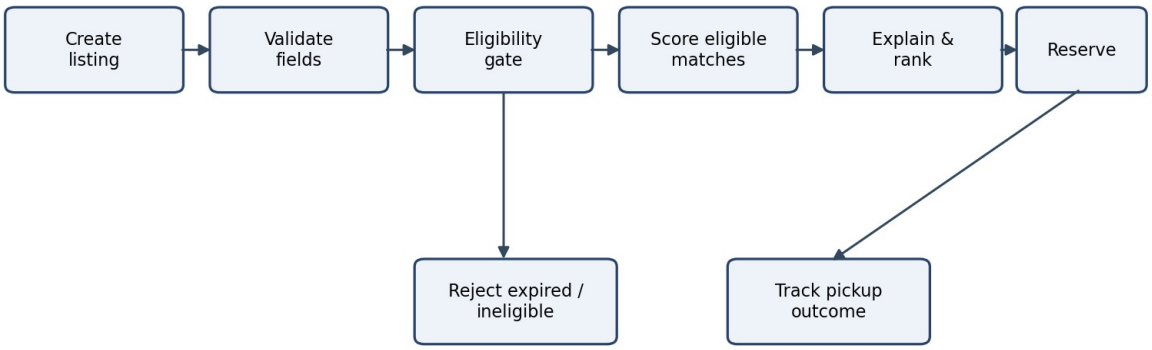


3.4 Workflow and UML/DFD Views

The workflow begins when a donor enters an offer. Required fields are validated, then the listing is checked against the recipient's profile and collection window. Ineligible pairs are excluded before scoring. The remaining candidates are ranked by score, with earlier deadlines and listing times used as tie-breakers. The recipient can review reasons before reserving a listing.

After reservation, collection status and outcome should be recorded. A valid lifecycle distinguishes a listing from an active reservation and from a completed, cancelled or missed collection. The state transition must prevent an already reserved or collected listing from being reserved again. This is a design requirement; the paper does not report that duplicate-reservation prevention has been tested.

Figure 3.2: End-to-end workflow



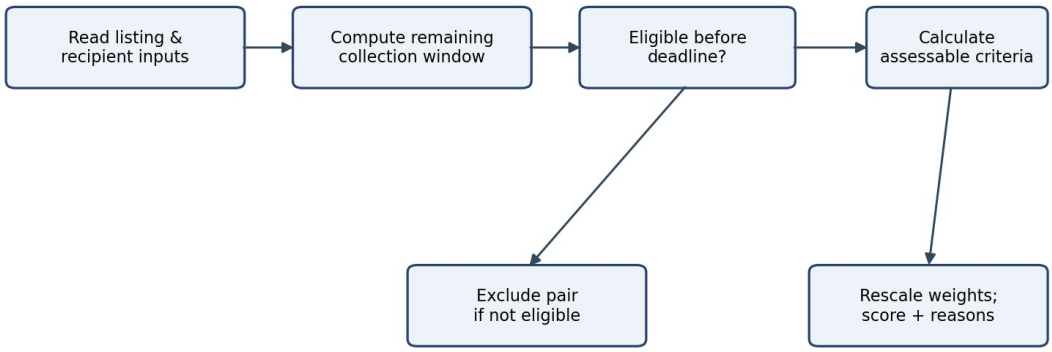
The workflow separates input validation and eligibility from scoring. Expired or otherwise ineligible pairs are excluded; only eligible candidates are ranked and explained. Reservation then connects matching to pickup-outcome tracking.

3.5 Algorithms

The ranking algorithm has two stages. Stage one is a hard eligibility gate: the listing must be open; its remaining collection window must be positive; a reliable travel-time estimate, if available, must be shorter than the remaining window; and a recipient maximum quantity of zero or less makes the pair ineligible. Invalid listing quantities and missing deadlines are rejected at entry.

Stage two scores eligible pairs. Urgency is always assessed. Category fit is assessed only when the recipient has a stated preference. Quantity fit is assessed only when quantities are available in matching units. Feasibility is assessed only when reliable travel time is available. Unassessable criteria are omitted and the remaining base weights are rescaled. The algorithm and tie-break rules in the paper are proposed specifications until checked against source code.

Figure 3.3: Eligibility and urgency-aware ranking



3.6 Data Flow

At context level, donors supply donation details and recipients supply their requirements. The application returns available listings, ranked recommendations and explanations. Reservation actions update status records, and pickup outcomes feed the analytics view. Administrator views summarise listings and records across the demonstration workspace.

At a more detailed level, the system validates inputs, builds eligible listing-recipient pairs, computes the set of assessable criteria, calculates a score and coverage, attaches explanations, sorts candidates, and records a reservation or outcome. In demo mode the state is browser-local. A multi-user cloud design would need transaction-safe reservation operations to avoid two recipients claiming the same donation concurrently.

3.7 System Modules

Donation input captures food name, category, quantity, unit, location, availability start, deadline, storage information and contact details. Discovery supports category filtering and sorting by soonest deadline, most recent listing or largest quantity. Recipient profiles store preferences, capacity and earliest collection time.

Matching and explanation calculate an auditable score and describe assessed and unassessed criteria. Reservation and pickup tracking represent progress and outcome. Analytics summarises records by period and offers a report download in the described interface. Administrator views provide a view of all donations. The module boundaries are conceptual and support maintenance and testing.

Table 3.2: Module description

Module	Inputs	Outputs / purpose
Donation input	Food details, quantity, unit, location, dates	Validated listing
Discovery	Listing records, category, sort choice	Filtered and ordered offers
Recipient profile	Category preference, capacity, collection start	Matching constraints
Matching engine	Listings and recipient profile	Eligible ranked pairs
Explanation	Criterion values and weights	Score breakdown and missing-data reasons
Reservation / pickup	Listing and recipient identifiers, status	Lifecycle and collection outcome
Analytics / admin	Stored listings, reservations, outcomes	Counts and impact summaries

3.8 Hardware Requirements

FoodLoop is a browser-based software prototype. No dedicated hardware or sensor network is specified in the supplied FoodLoop paper. The basic user environment is a device with a modern web browser and internet access to open the public demonstration. This is a usage requirement, not evidence of a deployed backend.

No motor, sensor network or telemetry hardware is specified in the supplied FoodLoop paper. The basic user environment is a device with a modern web browser and internet access to open the public demonstration. This is a usage requirement, not evidence of a deployed backend.

3.9 Software Requirements

The technology table in the conference paper records HTML, CSS and JavaScript as the stated client-side stack, with the code not inspected. The public demonstration is served from a Netlify address, which was observed once. Browser storage is stated for demo-mode persistence. Supabase Auth is mentioned in the sign-in interface, but the connection and shared database were not tested.

Maps and travel-time APIs, machine learning, external notifications and shared database security are not confirmed in the supplied evidence. They should be described as unimplemented, not found or future work, depending on the feature. The stack must be checked against source code before final technical claims are made.

Table 3.3: Software requirements and evidence status

Technology / service	Intended role	Evidence status
HTML, CSS, JavaScript	Client-side workspace	Page retrieved; source code not inspected
Netlify	Public demonstration hosting	Observed once
Browser storage	Demo-mode persistence	Stated on page; not tested
Supabase Auth	Cloud-mode sign-in	Stated in UI; connection not tested
PostgreSQL / RLS	Shared database and access control	Not evaluated
Maps / travel-time service	Collection feasibility	Not implemented in supplied evidence
Notifications / machine learning	Alerts or adaptive ranking	Not found / not claimed

3.10 Database Design

The paper's data model is a specification of records needed by the method, not a description of a verified deployed schema. Four logical entities are required: Listing, Recipient Profile, Reservation and Pickup Outcome. The Listing entity contains category, quantity and unit, availability start, collect-by deadline, location, status and listing time. The Recipient Profile entity contains preference, maximum quantity and unit, collection start and location.

Reservation records connect a listing to a recipient and record the reservation time and status. Pickup Outcome records the outcome, collected quantity and time. Separating collected quantity from listed quantity is important for reporting actual outcomes. In a shared deployment, one active reservation per listing should be enforced through database constraints or transactional logic, supported by row-level access policies [14], [15].

Figure 3.4: Logical data model (proposed schema)

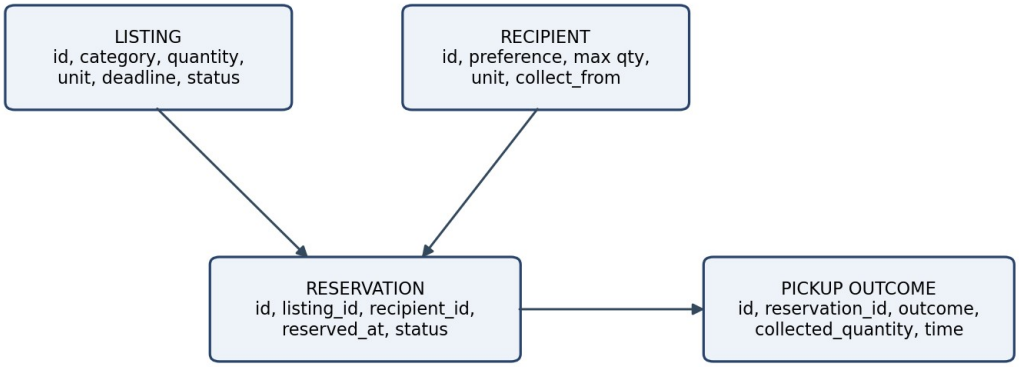


Table 3.4: Logical data entities and fields

Entity	Core fields	Purpose
Listing	id, category, quantity, unit, available_from, collect_by, location, status, listed_at	Eligibility and scoring inputs
Recipient Profile	id, preferred_category, max_quantity, unit, collect_from, location	Recipient constraints
Reservation	id, listing_id, recipient_id, reserved_at, status	Tracks active and historical claims
Pickup Outcome	id, reservation_id, outcome, collected_quantity, recorded_at	Records collected, missed or cancelled outcomes

3.11 Component Description

Validation checks required fields and data types. Quantity must be a positive numeric value with a unit, and the collection deadline must be present. Eligibility logic checks the listing status and remaining window. It also excludes a pair when a reliable travel-time estimate would consume the entire remaining window. If a maximum recipient quantity is zero or negative, the recipient cannot accept a positive quantity.

Scoring uses four base criteria and dynamically rescales weights for criteria that can be assessed. Explanation generation reports both assessed contributions and the reason a criterion was omitted. Sorting is deterministic when score and deadline tie, using listing time and identifier as further tie-breakers. Reservation management should enforce valid state transitions and avoid duplicate active claims.

3.12 User Interface Design

The public demonstration is described as a single-page workspace with Overview, Discover food, Saved food, Donate food, Smart matching, My reservations, Pickup tracking, Impact analytics, Admin dashboard and About sections. A role selector offers Recipient, Donor and Administrator views. Discovery supports a category filter and sorting options.

The interface should make the score, assessed coverage and reasons understandable without requiring a user to inspect source code. The design should distinguish sample records from real partner data and should clearly label the demo's local persistence. Actual application screenshots were not included in the supplied files, so this report does not fabricate screenshots or label conceptual illustrations as captured UI.

3.13 Feasibility Study

Technical feasibility is plausible for a small browser-based prototype because the described workflow relies on forms, client-side validation, arithmetic scoring and record views. However, the actual source code was not inspected, so implementation completeness cannot be certified from the current evidence. The scoring arithmetic is deliberately simple enough to test manually.

Operational feasibility depends on donors and recipient organisations entering accurate, timely information and following safe handling and collection practices. Economic feasibility cannot be quantified from the supplied materials because no cost study was provided. Legal and ethical feasibility requires compliance with applicable food-safety rules, privacy protections and partner consent. FoodLoop itself does not certify compliance.

3.14 Requirement Specification

Functional requirements specify what the system should do. Non-functional requirements address qualities such as transparency, reliability, privacy, usability and maintainability. These requirements are a basis for future verification; they should not be confused with evidence that each requirement is already met.

Table 3.5: Functional requirements

ID	Requirement	Priority
FR1	Capture donation details and validate required fields	High
FR2	Filter and sort available listings	High
FR3	Capture recipient preferences and capacity	High
FR4	Exclude ineligible listings before scoring	High
FR5	Calculate a score and assessed-weight coverage	High
FR6	Explain assessed and unassessed criteria	High
FR7	Record reservation and pickup outcome	High
FR8	Summarise outcomes in analytics views	Medium
FR9	Enforce role permissions in shared deployment	High for cloud

Table 3.6: Non-functional requirements

Category	Requirement
Transparency	Scores and reasons must be reproducible from inputs.
Data integrity	Reservation and outcome records must follow valid state transitions.
Privacy	Collect only necessary contact data and restrict access.
Security	Browser-local demo data must not be represented as secure shared storage.
Usability	Labels, deadlines, units and statuses should be understandable.
Maintainability	Eligibility, criterion functions and weights should be separately testable.
Reliability	Analytics should be computed from records rather than hard-coded totals.

CHAPTER 4

IMPLEMENTATION AND METHODOLOGY

4.1 Methodology

The methodology combines requirements analysis, a rule specification, a browser-based interface concept and a hand-checkable evaluation plan. The project begins by defining the information needed from a donor and recipient. It then separates hard eligibility from ranking so that urgency cannot make an impossible collection appear acceptable.

The conference paper describes a public demonstration page but explicitly states that it was not operated and its source code was not inspected. Consequently, implementation details below are categorised as visible in the UI, stated by the page, proposed in the paper or not evaluated. This evidence distinction is part of the technical method rather than a disclaimer added after the fact.

4.2 Donation Listing and Discovery

The donation form described by the demonstration collects a food name, category, quantity and unit, pickup location, availability start, collect-by deadline, storage information and contact details. Categories include produce, bakery, prepared meals, dairy, pantry and other. Quantity units include kilograms, items, portions, boxes and litres.

Discovery supports a category filter and sorting by soonest deadline, most recent listing or largest quantity. These functions help users narrow the set of offers before matching. The interface marks required fields, but actual validation behaviour has not been execution-tested in the supplied evidence. The required behaviour should be verified using empty, invalid and boundary-value inputs.

Table 4.1: Donation listing fields

Field	Purpose	Validation / note
Food name	Identify the offer	Required descriptive value
Category	Compare with recipient preference	One of the supported categories
Quantity and unit	Check recipient capacity	Positive numeric quantity; unit required
Pickup location	Tell recipients where collection occurs	Location is captured; routing not implemented
Available from	Prevent collection before availability	Used to determine usable start
Collect-by deadline	Enforce time window	Required; expired listings excluded
Storage information	Communicate handling details	Does not certify food safety
Contact details	Coordinate pickup	Personal data; share carefully

4.3 Matching and Prioritisation

Let a listing l have quantity Ql in unit ul, deadline dl, availability start al and category cl. Let a recipient r have optional preferred category cr, maximum quantity Qr in unit ur and collection start tr. Let tnow be the current time. The usable start is ts = max(tnow, al, tr), and the remaining window is Trem = dl − ts. H is the fixed urgency horizon and Ttrav is an optional reliable travel-time estimate.

Eligibility requires an open listing and Trem greater than zero. If reliable travel time exists, the pair is excluded when Ttrav is greater than or equal to Trem. A missing deadline or non-positive/non-numeric listing quantity is rejected at entry. A recipient maximum of zero or less makes the pair ineligible. Missing maximum quantity or a unit mismatch leaves quantity fit unassessed, rather than assuming a match.

Table 4.2: Criterion functions and base weights

Criterion	Base weight	Function / assessment condition
Urgency	0.40	furg = 1 − min(1, Trem/H); always assessed for eligible pair
Category fit	0.25	1 when listing category matches stated preference, otherwise 0; only assessed if preference exists
Quantity fit	0.20	min(1, Qr/Ql); only if both quantities exist and units match
Feasibility	0.15	(Trem − Ttrav)/Trem; only if reliable travel time exists

Scoring and assessed-weight coverage

The score for assessable criteria A is S(l,r) = 100 × Σk∈A w′k fk, where w′k = wk / Σj∈A wj. The assessed-weight coverage is C = 100 × Σk∈A wk. Because urgency is always assessed for an eligible pair, minimum coverage is 40%. Scores with different coverage should not be treated as strictly comparable.

4.3 Matching and Prioritisation (continued)

The reason shown for each assessed criterion should state the input values and the score points contributed, calculated as 100 × w′k fk. These points must sum to the total score within rounding. Unassessed criteria should be listed with their cause. This design ties explanation to the calculation and avoids a separate explanation process that could contradict the score.

After scoring, candidates are ordered by descending score, then earlier collect-by deadline, then earlier listing time, and finally listing identifier. The base weights are judgement-based, were not learned from data and have not been validated. Their usefulness must be tested against baseline orderings and reviewed with donors and recipients.

Table 4.3: Edge-case handling

Case	Proposed treatment
Expired listing or Trem ≤ 0	Ineligible; do not score
Missing deadline	Reject listing at entry
Quantity zero, negative or non-numeric	Reject listing at entry
No category preference	Category criterion unassessed
Missing / negative / unreliable travel time	Feasibility unassessed
Ttrav ≥ Trem	Ineligible; do not score
Only urgency assessable	Score on urgency; coverage 40%
Maximum quantity ≤ 0	Recipient-listing pair ineligible
Maximum quantity missing or unit mismatch	Quantity fit unassessed; rescale weights

4.4 Explainability

Explainability in FoodLoop is arithmetic transparency. A recommendation should state why it is ranked as it is, including how much each criterion contributes and which criteria could not be assessed. For example, if travel time is missing, the explanation should not imply that the recipient is nearby or that the donation is feasible based on distance.

Because the explanation is generated from the same criterion values and rescaled weights as the score, its contributions can be reproduced manually. This supports auditing, user trust and debugging. It does not prove that the chosen weights are fair, that the data are accurate, or that the top-ranked donation is always the best social outcome.

4.5 Reservation and Pickup Tracking

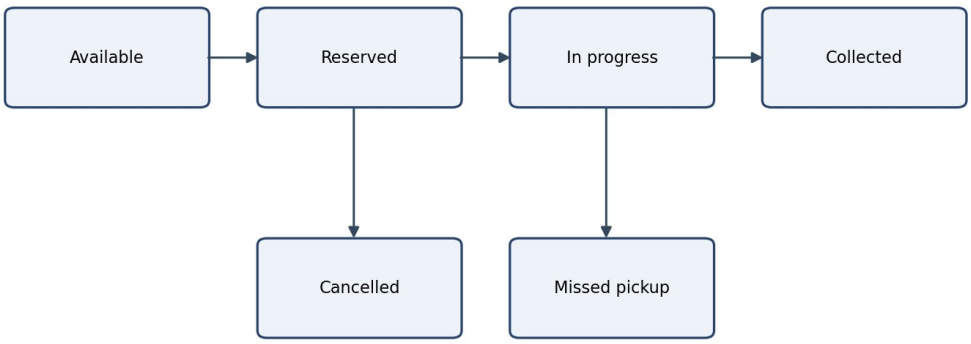
The described interface contains a reservation table with status filters and pickup tracking views for in-progress and completed collections. The design should record the reservation identifier, listing identifier, recipient identifier, reservation time and status. Pickup outcomes should separately record whether the collection was completed, missed or cancelled and the quantity actually collected.

State integrity is important. A reserved listing should not be reserved a second time, and a collected listing should not transition back to reserved. In a browser-only demo, these rules can be exercised within one device but do not establish safe concurrency between multiple users. A shared deployment would need transactional enforcement and access policies.

Figure 4.3: Reservation and pickup-outcome lifecycle

Conceptual lifecycle: Available → Reserved → In progress → Collected. Alternative terminal outcomes include Cancelled and Missed. The state transition rules are proposed requirements and must be tested against the actual implementation.

Figure 4.3: Reservation and pickup-outcome lifecycle



4.6 Analytics and Roles

The public demonstration describes impact analytics with a period filter and outcome panels, a downloadable report, and an administrator view of all donations. Analytics should be calculated from persisted records. For example, completed collections should be counted from completed pickup outcomes, and collected quantity should be summed from outcome records rather than assumed equal to the listed quantity.

The role selector offers Recipient, Donor and Administrator views. A selector in a demo interface does not itself establish secure permissions. In a cloud deployment, the system must authenticate users and enforce authorisation at the data layer, including row-level policies where appropriate. The supplied paper does not verify those controls.

4.7 Technology and Implementation Status

The public page was served from a Netlify address and was read on 16 October 2026. The conference paper says its source code was not inspected, so the exact implementation of validation, eligibility, scoring, reservation conflict prevention and data persistence remains unverified. The published base weights are reflected in the proposed specification, but the code-to-specification correspondence is unchecked.

HTML, CSS and JavaScript are listed as the client-side stack. Browser storage is described as the demo persistence mode. Supabase Auth is stated in the sign-in dialog, but connection and database behaviour

were not tested. This report therefore avoids claims of cloud deployment, role-secure persistence, external API integration, notifications or machine-learning implementation.

Table 4.4: Technology status

Feature / technology	Status	Implication
HTML / CSS / JavaScript	Unverified; page retrieved, code not inspected	Do not infer code-level implementation details
Netlify hosting	Observed once	Confirms public page availability at that time only
Browser storage	Stated, not tested	Device-local; not shared or secure
Supabase Auth	Stated; connection not tested	Cloud mode not evaluated
Database and RLS	Not evaluated	No shared-backend claims
Travel-time / map service	Not implemented in supplied evidence	Feasibility criterion often omitted
ML / notifications / external APIs	Not found in evidence	Future work if required

CHAPTER 5

RESULTS AND DISCUSSION

5.1 Evaluation Status

No test has been run for the conference paper, and no experimental results are reported. The public demonstration was read but not operated; its source code was not inspected. Therefore, this report provides expected outcomes and hand-computed examples only. These are not measured performance results and must not be represented as successful validation.

The evaluation plan includes eligibility edge cases, scoring arithmetic, missing-data behaviour, reservation conflicts, state transitions, local persistence, analytics consistency and sensitivity to weight changes. Executing these tests against the actual build is necessary before the project can claim functional correctness.

5.2 Worked Example

Consider a hypothetical listing with 120 minutes remaining, a fixed urgency horizon H of 12 hours, a category that matches the recipient preference, a listing quantity of 60 kg and a recipient maximum of 50 kg. Assume a reliable travel time of 20 minutes. This example is copied in substance from the supplied paper and is illustrative only.

The urgency value is 1 − 2/12 = 0.833. Category fit is 1. Quantity fit is 50/60 = 0.833. Feasibility is (120 − 20)/120 = 0.833. All four criteria are assessed, so the base weights apply without rescaling. The approximate score contributions are 33.3, 25.0, 16.7 and 12.5 points, producing a total score of 87.5 with 100% coverage.

Table 5.1: Worked example values

Criterion	Value	Base weight	Contribution (approx.)
Urgency	0.833	0.40	33.3
Category fit	1.000	0.25	25.0
Quantity fit	0.833	0.20	16.7
Feasibility	0.833	0.15	12.5
Total	—	1.00	87.5; coverage 100%

5.2 Worked Example (continued)

If travel-time data are absent, feasibility is not assessed. The remaining base-weight sum is 0.85, so the assessed weights are rescaled. The paper’s worked example reports a score of 88.2 at 85% coverage. If travel time and category preference are both absent, the paper’s test plan expects a score of 83.3 at 60% coverage.

These values illustrate why score coverage must accompany the score. A candidate scored on fewer criteria may appear numerically higher after rescaling, even though less of the intended criteria were assessed. For operational use, users should see both the score and coverage and should avoid treating different coverage levels as directly comparable.

5.3 Test Plan

The tests below are proposed and have not been executed. T1 and T2 validate the eligibility gate. T3–T5 verify the worked example and missing-data rescaling. T6–T10 check invalid inputs, quantity fit, urgency monotonicity and arithmetic consistency. T11–T12 check reservation integrity. T13 checks the stated browser-storage behaviour. T14 checks analytics consistency from synthetic outcomes. T15 evaluates sensitivity to changes in the base weights.

Table 5.2: Test cases and expected outcomes

ID	Input / action	Expected outcome
T1	Trem = 0 or negative	Pair not shown
T2	Ttrav = Trem	Pair not shown
T3	Worked example with all criteria	Score 87.5; contributions sum to score; 100% coverage
T4	T3 without travel time	Score 88.2; 85% coverage; feasibility unassessed
T5	No travel time or category preference	Score 83.3; 60% coverage
T6	Recipient max ≤ 0 / unit mismatch	≤ 0 ineligible; mismatch makes quantity unassessed
T7	Quantity 0 or −5; missing deadline	Listing rejected at entry
T8	Listing quantity ≤ recipient maximum	Quantity fit = 1; no shortfall penalty
T9	Same listing with smaller remaining window	Urgency value increases
T10	Any scored pair	Displayed points sum to score within rounding
T11	Second reservation of reserved listing	Rejected
T12	Collected listing set back to reserved	Invalid transition rejected
T13	Reload same browser; open another browser	Data remains in first profile; absent in second
T14	Synthetic outcome records	Analytics counts match records
T15	Change weights and rescale	Record top-ranked listing changes

5.4 Expected Outcomes

If the specification is implemented as written, expired or infeasible pairs should not appear in ranked results. Missing criteria should be labelled as unassessed and the remaining weights should be rescaled. Explanations should add up to the displayed score. A smaller remaining window should increase urgency, while a travel time that consumes the entire window should exclude the pair.

Reservation integrity and analytics correctness are equally important. A second active reservation should be rejected, and the dashboard should count outcomes from the stored records. These are expected behaviours from the design, not observed outcomes. The report intentionally does not include fabricated screenshots, measured ranking accuracy, user counts or field collection rates.

5.5 Discussion

The central design choice is to use eligibility as a gate and urgency as a ranking signal. Urgency alone would promote a nearly expired offer even when it could not be collected in time. The gate prevents that outcome when the required timing data are available. It also means that an offer may be excluded under strict rules even if a recipient might conceivably collect it under unusual circumstances.

Missing-data handling avoids silently rewarding unknowns. However, rescaled scores with different coverage are not directly comparable. Urgency and feasibility can also pull in opposite directions: a shorter window raises urgency while reducing the available slack for travel. A sensitivity analysis and comparisons with first-come and soonest-deadline-first baselines are needed before claims of improved outcomes can be made.

CHAPTER 6

FOODLOOP WORKSPACE AND DEMONSTRATION

6.1 Overview

The public demonstration presents FoodLoop as a single-page workspace. Its described sections include Overview, Discover food, Saved food, Donate food, Smart matching, My reservations, Pickup tracking, Impact analytics, Admin dashboard and About. The interface provides role views for recipients, donors and administrators.

The conference paper notes that the page was retrieved but not operated. Accordingly, this chapter documents the stated layout and intended interactions. It does not claim a successful end-to-end demonstration or verified data persistence. The supplied files contain no actual captured FoodLoop application screenshots that could be used as implementation evidence.

6.2 Main Views

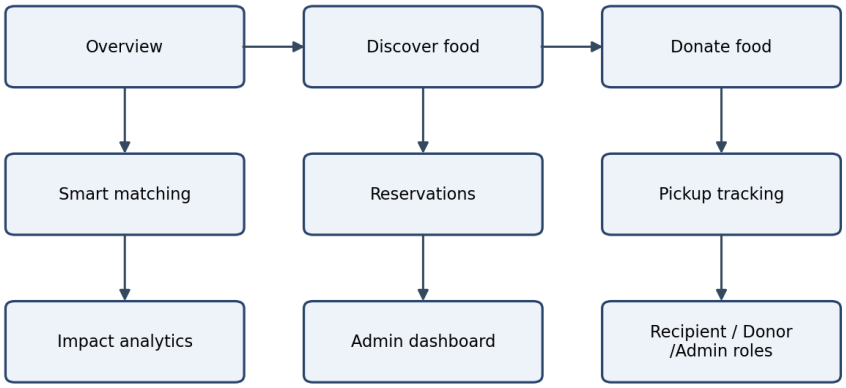
Overview can orient a user to available records and activity. Discover food allows category filtering and sorting by deadline, listing time or quantity. Donate food captures structured surplus information. Smart matching accepts a sample or custom recipient profile and displays a prioritised list described as rule-based. Reasons are intended to clarify the ranking.

My reservations and Pickup tracking provide views of claimed donations and collection progress. Impact analytics summarises records over a selected period and offers a report download. The administrator view provides an overview of donations. These are described UI capabilities; actual behaviour and the integrity of underlying records remain to be tested.

Figure 6.1: FoodLoop demonstration sections (conceptual view)

Conceptual representation of the described workspace; this is not an application screenshot.

Figure 6.1: FoodLoop workspace views (conceptual)



6.3 Demo Data and Storage

The demonstration page labels sample records as samples and states that changes remain in the browser. Browser storage is specific to one browser profile on one device. It is not shared between users and is not a secure data store, so it cannot support a multi-user deployment on its own.

The page's sign-in dialog describes a cloud mode using Supabase Auth. However, the paper does not evaluate a database behind that mode, and no connection, database schema or row-level access policy is confirmed. A production-oriented implementation would need shared persistence, authentication, authorisation, concurrency control, backups and an operational plan for incidents.

6.4 Safety, Privacy and Fairness

Food safety remains the responsibility of donors and recipient organisations. The paper cites the Food Safety and Standards (Recovery and Distribution of Surplus Food) Regulations, 2019, which establish responsibilities and requirements for surplus-food recovery and distribution [16]. FoodLoop does not inspect, certify or guarantee that listed food is safe, and its fields are not claimed to satisfy all regulatory requirements.

Contact information is personal data. A shared deployment should collect only necessary information, obtain appropriate consent, restrict access and follow applicable data-protection law. Browser storage is not secure shared storage. Ranking may also favour recipients who check the platform frequently or have better connectivity or transport. Distribution of collected food should be monitored for fairness, with partner consent and institutional approval where required.

CHAPTER 7

CONCLUSION AND FUTURE WORK

7.1 Conclusion

FoodLoop presents a web-based design for coordinating surplus-food donations through structured listings, recipient matching, urgency-aware ranking, score explanations, reservations, pickup tracking and analytics. Its proposed method applies eligibility rules before ranking, excludes criteria that cannot be assessed, rescales the remaining weights, reports coverage and generates reasons from the score calculation.

The project's contribution is limited to this auditable combination of design choices. Digital donation platforms, volunteer recommenders and allocation models already exist in the literature. The supplied paper reports no experiment, and the correspondence between the proposed method and deployed code has not been verified. FoodLoop's usefulness therefore remains to be demonstrated through implementation checks and tests.

7.2 Achievement of Project Objectives

The project materials establish that a public demonstration page exists and describes the main workflow. They do not establish that the full system passed tests. The following table therefore records objective status in evidence-based terms, avoiding a blanket claim of completion.

Table 7.1: Objectives and evidence status

Objective	**Evidence-based status**
Structured donation listing	Fields visible/described in public UI; not tested
Discovery and sorting	Controls described; not tested
Recipient-specific matching	Matching view seen; logic not inspected
Eligibility and missing-data rules	Specified in paper; code correspondence unverified
Explainable score	Weights/reasons described; execution not verified
Reservation and pickup tracking	Views described; workflow not tested
Analytics	Panels/report download described; not tested
Secure shared deployment	Not evaluated
Measured effectiveness	No experiment conducted

7.3 Limitations

The primary limitations are the absence of an evaluation, judgement-based unvalidated weights, unverified correspondence between specification and code, device-local demo storage, unavailable verified travel-time data, and a targeted rather than systematic literature review. The demonstration page was read but not operated and source code was not inspected.

Scores with different assessed coverage are not strictly comparable. The local ranking method does not solve a global allocation problem or guarantee equity across recipient organisations. Food safety, donor verification, reliable routing, notification delivery and secure multi-user permissions are outside the verified current scope.

7.4 Future Scope

First, inspect the source code and compare the implemented ranking behaviour with the formal specification. Execute the test plan, capture genuine application screenshots and report the observed results. Second, evaluate shared cloud storage, authentication, row-level access control and transactional reservation rules. Third, add reliable travel-time estimates only when the data source is trustworthy and transparent.

Fourth, set and review weights with donor and recipient input. Fifth, compare FoodLoop against first-come and soonest-deadline-first baselines, then consider assignment-based or learning-based methods when suitable data are available. Sixth, run a consent-based field pilot with partner organisations, measuring collection rate, time to collection, quantity collected, missed pickups and fairness across recipients.

7.5 Summary

FoodLoop is a promising but unvalidated prototype concept. Its strongest design feature is the explicit connection between eligibility, urgency scoring, missing-data coverage and explanations. The next milestone is not to claim impact prematurely, but to verify implementation, execute the test plan, strengthen data protection and evaluate outcomes with real partners under appropriate safeguards.

REFERENCES

[1] J. Gustavsson, C. Cederberg, U. Sonesson, R. van Otterdijk, and A. Meybeck, Global Food Losses and Food Waste: Extent, Causes and Prevention. Rome, Italy: FAO, 2011.

[2] United Nations Environment Programme, Food Waste Index Report 2024: Think Eat Save – Tracking Progress to Halve Global Food Waste. Nairobi, Kenya: UNEP, 2024.

[3] F. D. Oroski and J. M. da Silva, "Understanding food waste-reducing platforms: A mini-review," Waste Management & Research, vol. 41, no. 4, pp. 816–827, 2023, doi: 10.1177/0734242X221135248.

[4] FAO AgriTech Observatory, "FoodCloud." [Online]. Available: https://agritechobservatory.review.fao.org/en/foodcloud. [Accessed: Oct. 16, 2026].

[5] Z. R. Shi, L. Lizarondo, and F. Fang, "A recommender system for crowdsourcing food rescue platforms," in Proc. Web Conf. 2021 (WWW '21), Ljubljana, Slovenia, Apr. 2021, pp. 857–865, doi: 10.1145/3442381.3449787.

[6] Z. R. Shi, Z. S. Wu, R. Ghani, and F. Fang, "Bandit data-driven optimization for crowdsourcing food rescue platforms," in Proc. AAAI Conf. Artificial Intelligence, vol. 36, no. 11, 2022, pp. 12154–12162, doi: 10.1609/aaai.v36i11.21475.

[7] D. Gale and L. S. Shapley, "College admissions and the stability of marriage," The American Mathematical Monthly, vol. 69, no. 1, pp. 9–15, 1962.

[8] H. W. Kuhn, "The Hungarian method for the assignment problem," Naval Research Logistics Quarterly, vol. 2, no. 1–2, pp. 83–97, 1955, doi: 10.1002/nav.3800020109.

[9] I. Sengul Orgut, J. Ivy, R. Uzsoy, and J. R. Wilson, "Modeling for the equitable and effective distribution of donated food under capacity constraints," IISE Transactions, vol. 48, no. 3, pp. 252–266, 2016.

REFERENCES (CONTINUED)

[10] T. L. Saaty, The Analytic Hierarchy Process: Planning, Priority Setting, Resource Allocation. New York, NY, USA: McGraw-Hill, 1980.

[11] M. K. Lee et al., “WeBuildAI: Participatory framework for algorithmic governance,” Proceedings of the ACM on Human-Computer Interaction, vol. 3, no. CSCW, Art. 181, pp. 1–35, Nov. 2019, doi: 10.1145/3359283.

[12] Y. Koren, R. Bell, and C. Volinsky, “Matrix factorization techniques for recommender systems,” Computer, vol. 42, no. 8, pp. 30–37, Aug. 2009, doi: 10.1109/MC.2009.263.

[13] FoodLoop, “FoodLoop — Good food, back in circulation,” demonstration web page. [Online]. Available: https://foodloop-community-rescue.netlify.app. [Accessed: Oct. 16, 2026].

[14] Supabase, “Row Level Security.” [Online]. Available: https://supabase.com/docs/guides/database/postgres/row-level-security. [Accessed: Oct. 16, 2026].

[15] The PostgreSQL Global Development Group, “PostgreSQL documentation: Row security policies.” [Online]. Available: https://www.postgresql.org/docs/current/ddl-rowsecurity.html. [Accessed: Oct. 16, 2026].

[16] Food Safety and Standards Authority of India, “Food Safety and Standards (Recovery and Distribution of Surplus Food) Regulations, 2019,” The Gazette of India, Extraordinary, Part III, Sec. 4, no. 278, New Delhi, Jul. 30, 2019.

APPENDIX

Table A.1: Notation and configuration variables

A.1 Notation and configuration variables

Symbol / variable	Meaning
Ql, ul	Listing quantity and unit
dl	Collect-by deadline
al	Availability start
cl	Listing food category
Qr, ur	Recipient maximum quantity and unit
cr	Optional recipient preferred category
tr	Recipient collection start
tnow	Current time
ts	Usable collection start = max(tnow, al, tr)
Trem	Remaining collection window = dl − ts
Ttrav	Optional reliable travel-time estimate
H	Fixed urgency horizon
wk	Base weight for criterion k
fk	Value of assessed criterion k
A	Set of assessable criteria
S	Final score on a 0–100 scale
C	Assessed-weight coverage percentage

A.2 Evidence and submission checklist

- Replace any placeholder institutional details only after confirmation by the department.
- Obtain authorised signatures and official seals through the institution; none are fabricated in this document.
- Capture genuine screenshots from the current FoodLoop application and label demo data clearly.
- Inspect source code and verify whether the implemented scoring logic matches the proposed algorithm.
- Run the listed tests and replace expected outcomes with actual results only after execution.
- Verify cloud configuration, authentication, database persistence and access rules before describing them as implemented.
- Review food-safety, privacy and partner-consent requirements before any field pilot.

This appendix records the checks needed before a final institutional submission. It does not substitute for implementation evidence or official academic approval.